1. **Minting and Burning LP Tokens**

Question: Which address(es) should be allowed to mint/burn the LP tokens?

Answer: Only the core DEX.sol smart contract should be granted the authority to mint and burn LP tokens. LP tokens represent a proportional claim on the total underlying reserves of Token A and Token B in the pool. If an arbitrary user were allowed to mint LP tokens, they could instantly generate a massive fake share of the pool and use it to drain the actual Token A and Token B reserves during a withdrawal.

- **Code Implementation:** In the code, LPToken.sol imports OpenZeppelin's Ownable. The deployer of the LPToken becomes the "owner". Because DEX.sol deploys LPToken.sol within its constructor, the DEX contract itself is the sole owner. The mint and burn functions are protected by the onlyOwner modifier, ensuring only the DEX can execute them during the addLiquidity and removeLiquidity functions.

2. Leveling the Playing Field

Question: In what way do DEXs level the playing ground between a powerful and resourceful trader and a lower resource trader?

Answer: Traditional order-book exchanges inherently favor High-Frequency Trading (HFT) firms and institutional investors who can colocate their servers physically close to the exchange's data centers, gaining microsecond latency advantages to execute trades first. An AMM DEX eliminates the order book entirely. Instead of matching buyers and sellers, everyone trades against a deterministic algorithm ($x * y = k$). Furthermore, the blockchain's block time (e.g., ~12 seconds on Ethereum) acts as a great equalizer. Because all transactions are batched into discrete blocks, the microsecond hardware advantages of institutional traders are completely neutralized.

3. Miner/Validator Exploitation (MEV)

Question: Suppose there are many transaction requests to the DEX sitting in a miner's mempool. How can the miner take undue advantage of this information? Is it possible to make the DEX robust against it?

Answer: Because the mempool is public, a miner (or validator) can see pending transactions before they are confirmed. If a validator sees a massive buy order that will push the price of Token B up, they can take undue advantage by executing a "Sandwich Attack." The validator places their own buy order just *before* the user's transaction (buying it cheap), lets the user's massive trade push the price up, and then places a sell order just *after* the user's transaction (selling high for an instant, risk-free profit).

- **Making it Robust:** To protect against this, production DEXs implement a "slippage tolerance" parameter (often called amountOutMin). When a user submits a swap, they define the absolute minimum amount of tokens they are willing to accept. If a validator manipulates the price in the mempool so that the output falls below this minimum threshold, the user's transaction simply reverts, denying the validator their profit.

4. Gas Fees and Economic Viability

Question: Every function call needs gas. How does gas fees influence economic viability of the entire DEX and arbitrage?

Answer: Gas fees introduce a fixed operating cost to every transaction. For retail traders, if network gas fees spike (e.g., $50 per swap), small trades become economically unviable because the fee might cost more than the tokens being swapped. For arbitrage, gas completely alters the profit equation. An arbitrage opportunity is only mathematically viable if: Gross Profit from Trade Discrepancy > Total Gas Cost for All Routing Steps. If an arbitrageur spots a $10 price difference between DEX 1 and DEX 2, but executing the two required swaps costs $15 in gas, the "opportunity" will result in a net loss.

5. Gas Fees as an Undue Advantage

Question: Could gas fees lead to undue advantages to some transactors over others? How?

Answer: Yes. Blockchains utilize a priority fee market (often called "bribing" the validators). When a highly profitable arbitrage opportunity appears, multiple bots and institutional traders will spot it simultaneously. A wealthy, high-resource trader can afford to submit the transaction with a massive priority gas fee. Validators, incentivized by profit, will order the transactions inside the block so that the highest-paying transaction executes first. This creates a "gas war," allowing well-funded transactors to consistently beat lower-resource retail transactors to profitable opportunities.

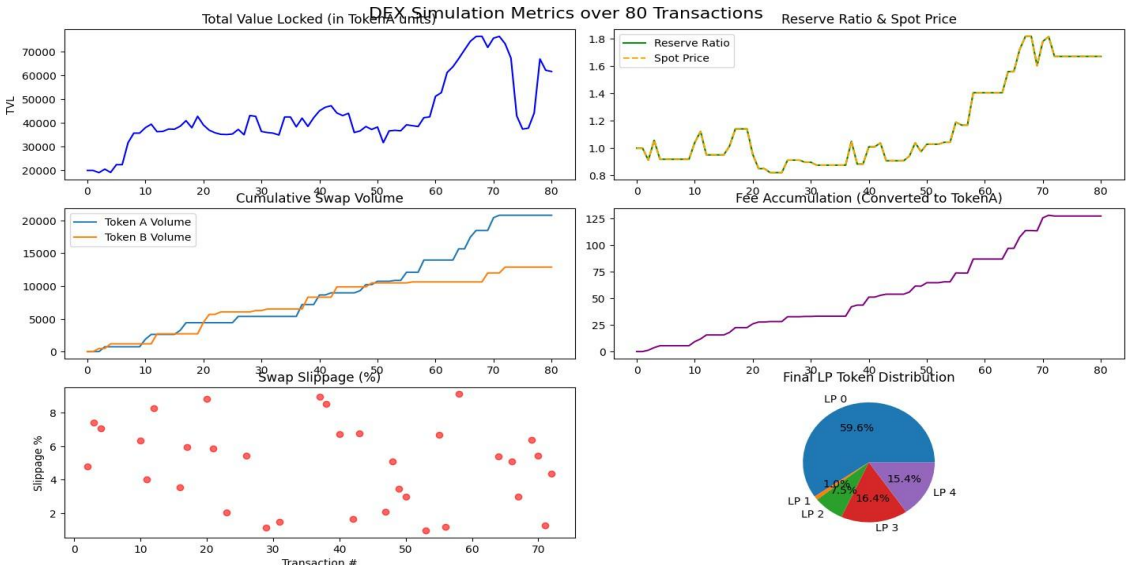
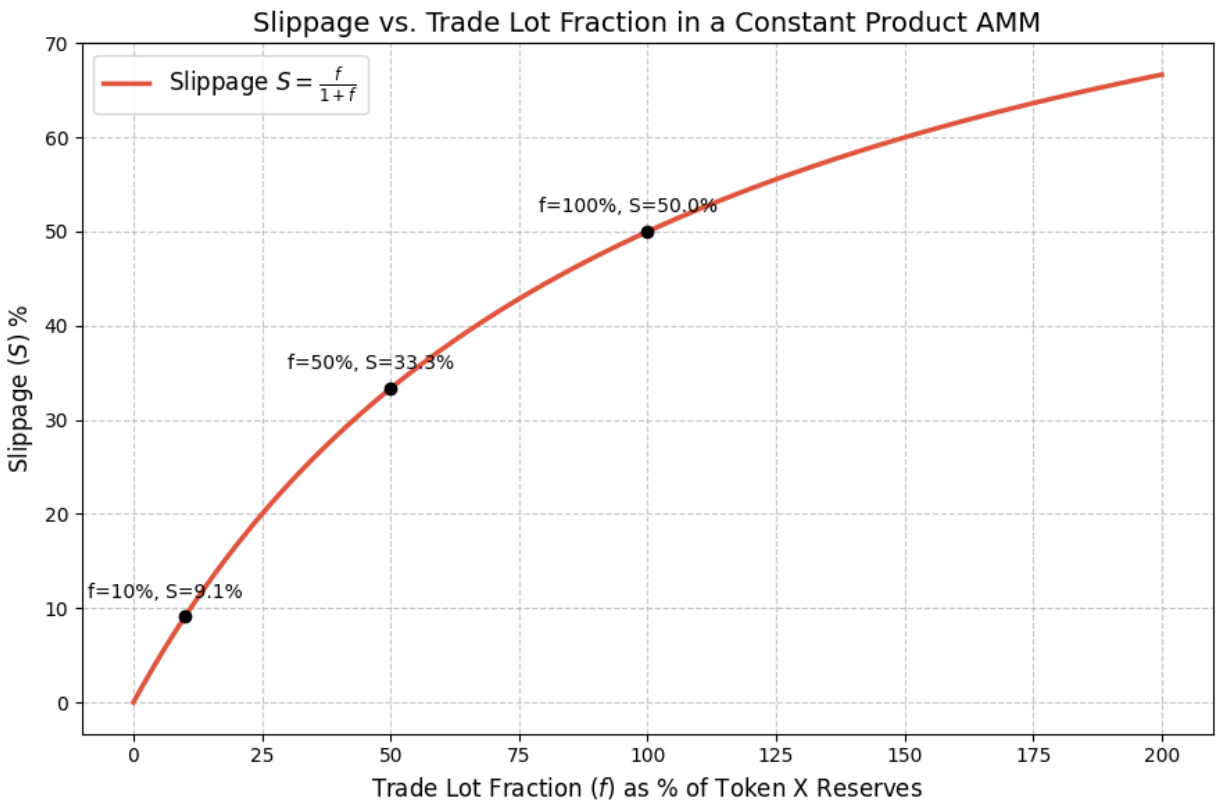
6. Minimizing Slippage

Question: What are the various ways to minimize slippage in a swap?

Answer: Slippage is the difference between the expected price based on current reserves and the actual executed price. It can be minimized by:

1. **Using High-Liquidity Pools:** The deeper the reserves of x and y, the less a trade of size Δx will impact the ratio.
2. **Trade Splitting:** Breaking a massive trade into several smaller trades over time allows arbitrageurs to bring the pool price back to the market equilibrium between your trades, resulting in a better average price.
3. **Setting Strict Slippage Tolerances:** As mentioned in Question 3, hardcoding a minimum acceptable output in the transaction ensures you never suffer slippage beyond your predefined comfort zone.

7. Slippage vs. Trade Lot Fraction

Question: Plot how slippage varies with the "trade lot fraction" for a constant product AMM.

Answer: